

Attention Via Convolutional Nearest Neighbors

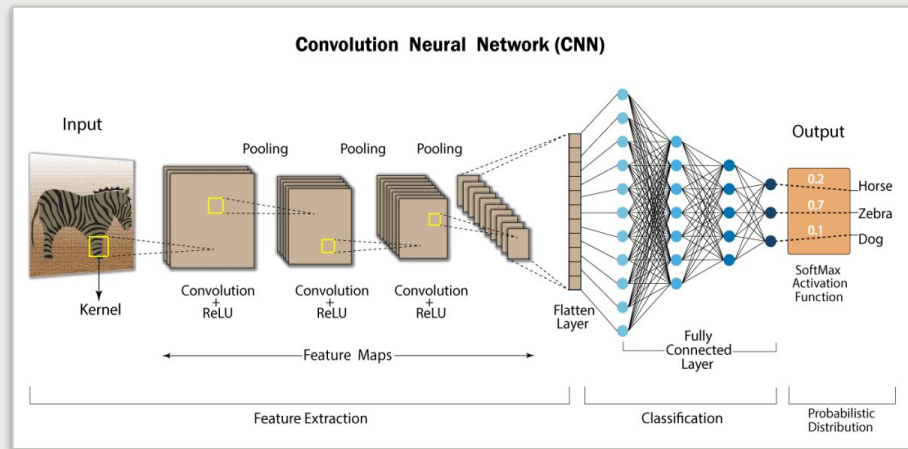
Mingi Kang Bowdoin 2026
Jeová Farias Sales Rocha Neto

Senior Honors Presentation

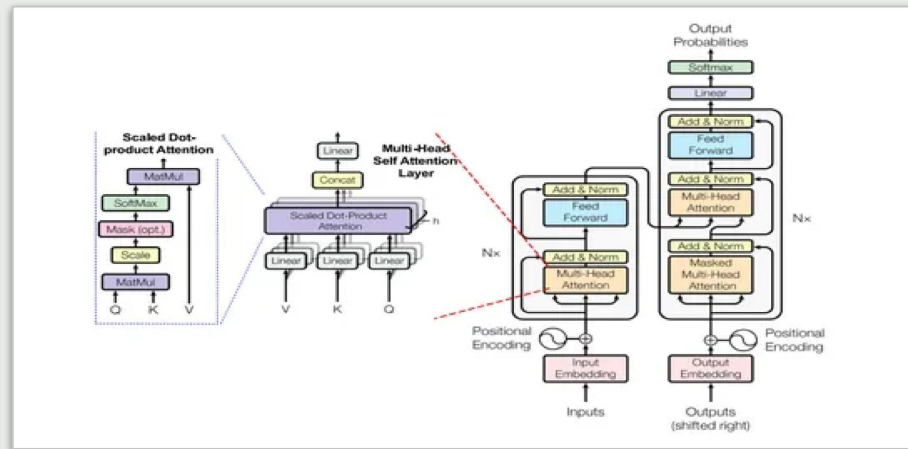
Motivation

CNNs and Transformers are the most used architectures across vision and language tasks.

But, how do their core operations: convolution and self attention, relate to each other?



Convolutional Neural Network (CNN)



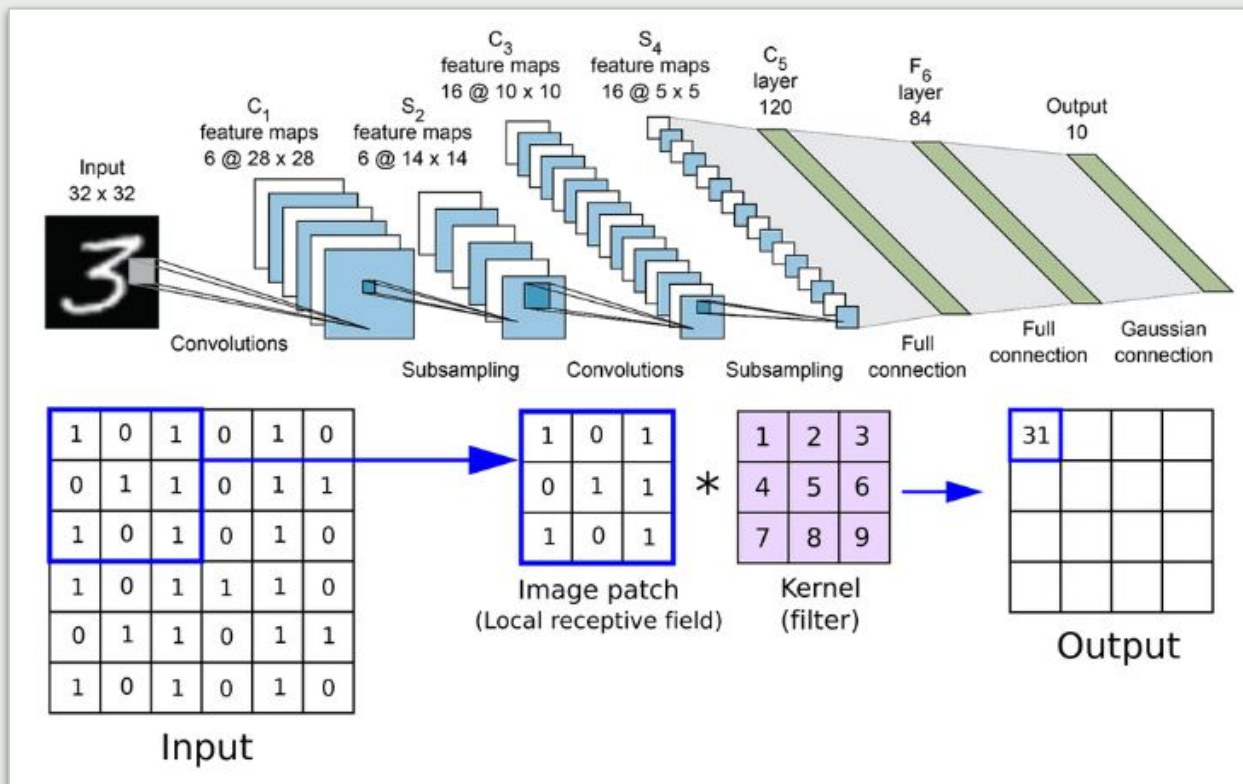
Transformer Architecture

Convolution

Convolutions captures **local spatial structure** by design.

Kernels slide across the input, learning patterns within a fixed neighborhood of pixels.

This **spatial locality bias** makes convolution powerful for images, videos, and other data with strong local correlations.

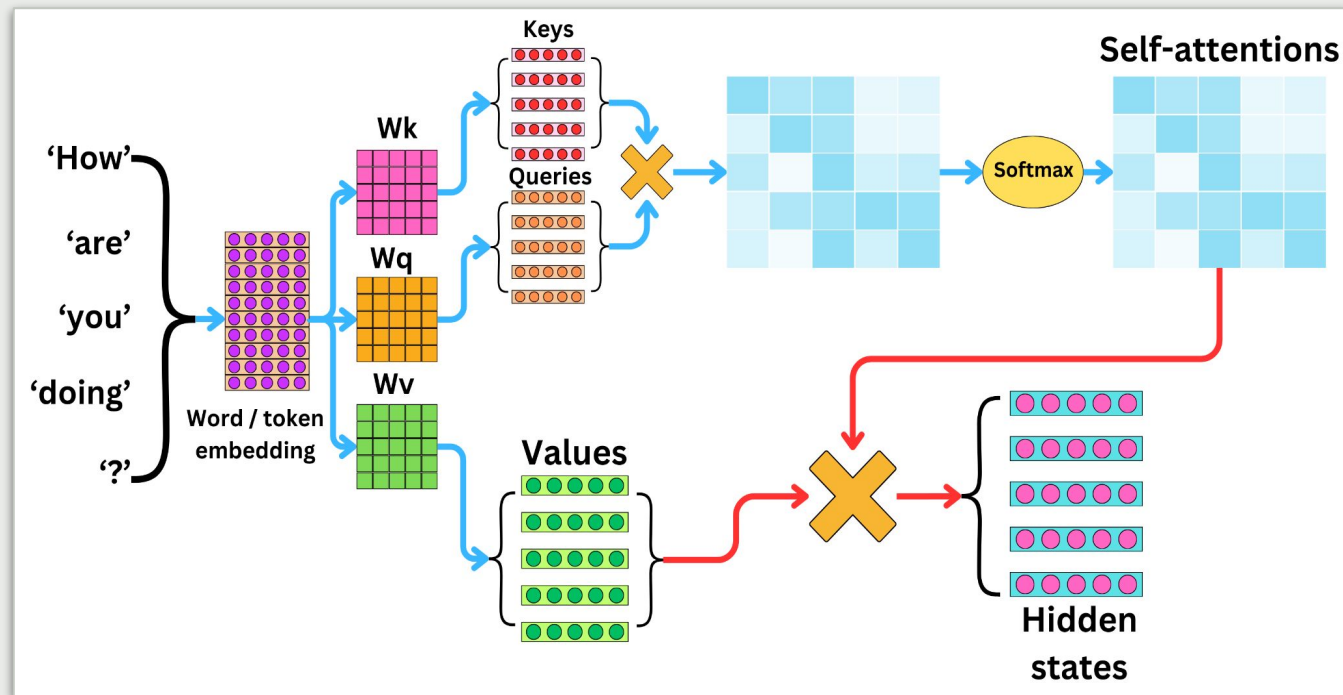


Self-Attention

Self-Attention (Vaswani et al. 2017) transforms inputs into Query (Q), Key (K), and Value (V).

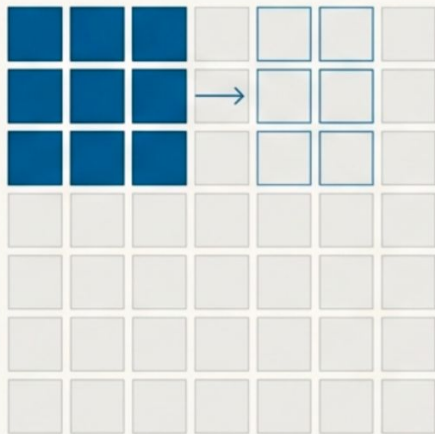
Attention Weights determines which positions to attend to based on learned feature similarity.

This enables **global receptive fields** where each token can aggregate information from any other position.



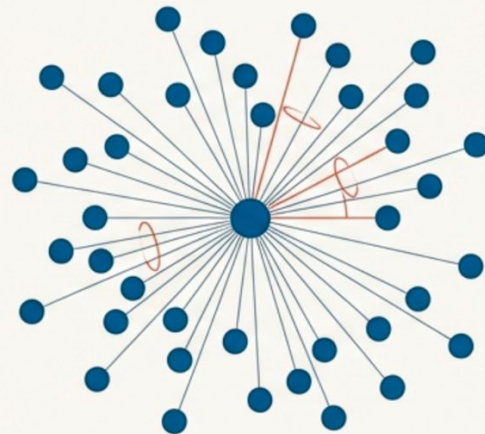
CNNs vs. Vision Transformers

Convolutional Neural Networks



- **Mechanism:** Local Spatial Aggregation
- **Neighbor Selection:** Spatial Proximity (Fixed Grid)
- **Strengths:** Spatial inductive biases, efficiency
- **Example Architectures:** VGG, ResNet

Vision Transformers



- **Mechanism:** Global Feature Aggregation
- **Neighbor Selection:** Feature Similarity (All-to-all)
- **Strengths:** Captures long-range dependencies
- **Example Architectures:** ViT, Swin

Convolutional Nearest Neighbor (ConvNN)

Unified Framework that formalizes the connection between convolution and attention through the **k-nearest neighbors** principle from machine learning.

ConvNN Framework

01

Similarity Computation

How do we measure similarity between pixels (features)?

Similarity Metrics:

- Cosine Similarity
- Euclidean distance
- Dot product

Operation:

- Convolution: Spatial proximity
- Attention: Feature similarity

02

Neighbor Selection

How do we identify the most relevant neighbors?

Selection Process:

- Compute $S = XX^T$
- k-max: top-k scores
- k-argmax: top-k indices

Result: k neighbors per pixel across all positions.

03

Weighted Aggregation

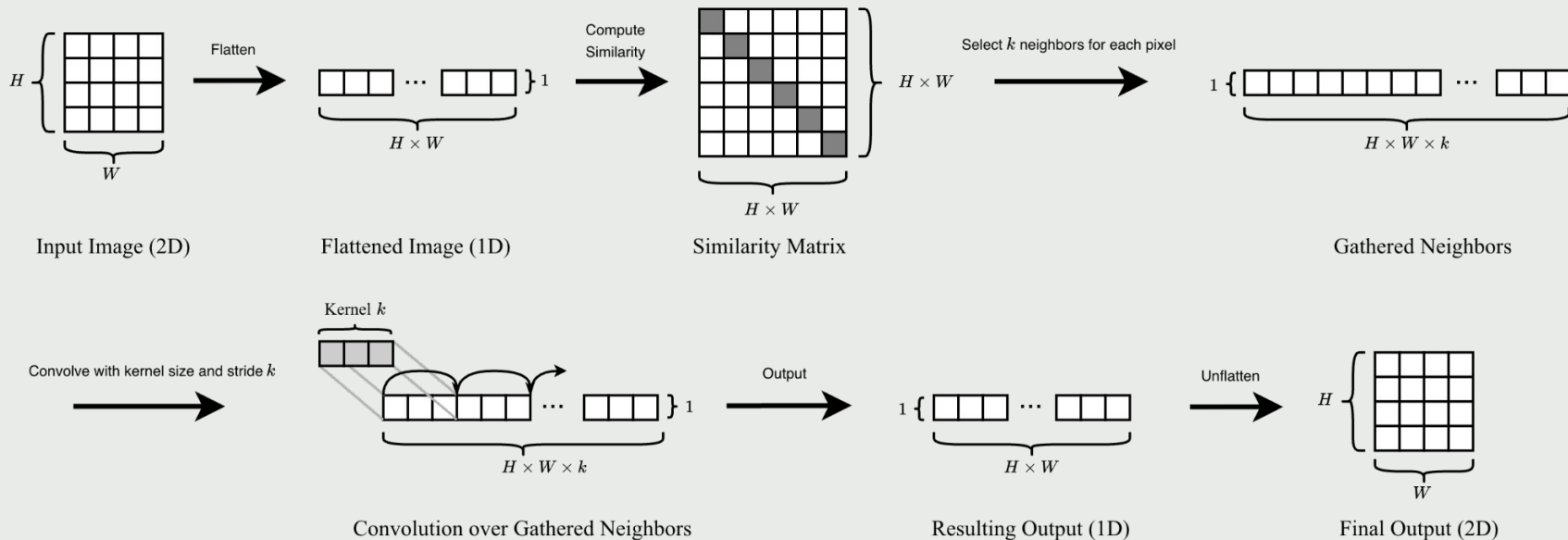
How do we aggregate information from selected neighbors?

Aggregation Options:

- Standard Convolution
- Depthwise Convolution
- Depthwise-Separable Convolution

Applied to concatenated neighborhood matrices
(stride = k, kernel = k)

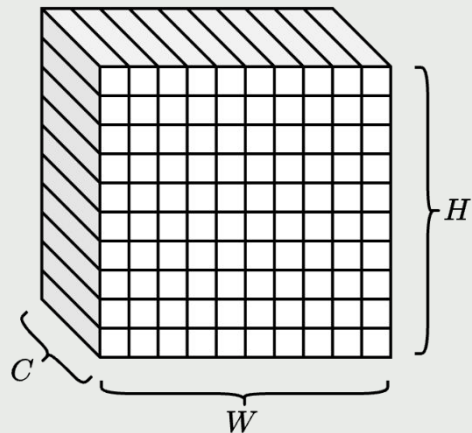
ConvNN Visualized



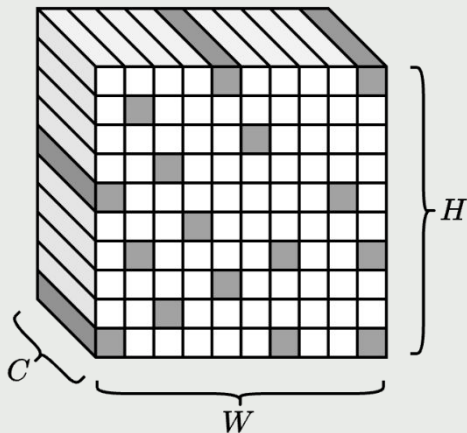
Full Convolutional Nearest Neighbors Visualized

*Channel dimensions are omitted in diagram for visual clarity.

Sparse Neighbor Search

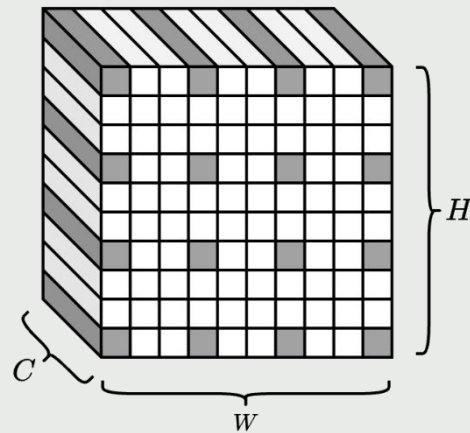


(a) Input Tensor



(b) After Random Sparsification

$$r = 16$$



(c) After Spatial Sparsification

$$r = 16, r' = 4$$

k-NN search complexity:

$$O(n^2 \log n) \rightarrow (nr \log r)$$

Generalization of Convolution

Traditional Convolution

$$S = X_{\text{coord}} X_{\text{coord}}^{\top}$$

$$I_i = k\text{-argmax}_k(S)$$

$$X_{nn,i} = X[I_i, :] \in \mathbb{R}^{k \times c}$$

$$X_{nn} \in \mathbb{R}^{(kn) \times c}$$

$$Y = \text{Conv1d}(X_{nn}) \in \mathbb{R}^{n \times c'}$$

ConvNN

$$S = X X^{\top}$$

$$I_i = k\text{-argmax}_k(S)$$

$$X_{nn,i} = X[I_i, :] \in \mathbb{R}^{k \times c}$$

$$X_{nn} \in \mathbb{R}^{(kn) \times c}$$

$$Y = \text{Conv1d}(X_{nn}) \in \mathbb{R}^{n \times c'}$$

Generalization of Attention

ConvNN-Attention

$$Q = f_Q(X), K = f_K(X), V = f_V(X)$$

$$S = QK^\top$$

$$I_i = k\text{-argmax}_k(S)$$

$$s_i = k\text{-max}_k(S)$$

$$X_{nn,i} = \text{softmax}(s_i)X[I_i, :] \in \mathbb{R}^{k \times c}$$

$$X_{nn} \in \mathbb{R}^{(kn) \times c}$$

$$Y = \text{DepthwiseConv1d}(X_{nn}) \in \mathbb{R}^{n \times c'}$$

Self-Attention

$$Q = f_Q(X), K = f_K(X), V = f_V(X)$$

$$S = QK^\top$$

$$I_i = k\text{-argmax}_k(S)$$

$$s_i = k\text{-max}_k(S)$$

$$X_{nn,i} = \text{softmax}(s_i)V[I_i, :] \in \mathbb{R}^{n \times c}$$

$$X_{nn} \in \mathbb{R}^{n^2 \times c}$$

$$Y = \text{UnitDepthwiseConv1d}(X_{nn}) \in \mathbb{R}^{n \times c}$$

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Experiments

Datasets: CIFAR-10 and CIFAR-100 (Krizhevsky et al., 2009)

- 60,000 images (32x32 RGB)
- 10 classes (CIFAR-10) / 100 classes (CIFAR-100)
- Train/test split: 50,000 / 10,000
- **No data augmentation** to ensure fair baseline comparison

Architectures:

- **VGG-11:** Convolutional Baseline (32x32 RGB)
- **ViT-Tiny:** Transformer Baseline (224x224 RGB)

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



VGG Architecture

VGG-11 Architecture (Simonyan & Zisserman, 2015)

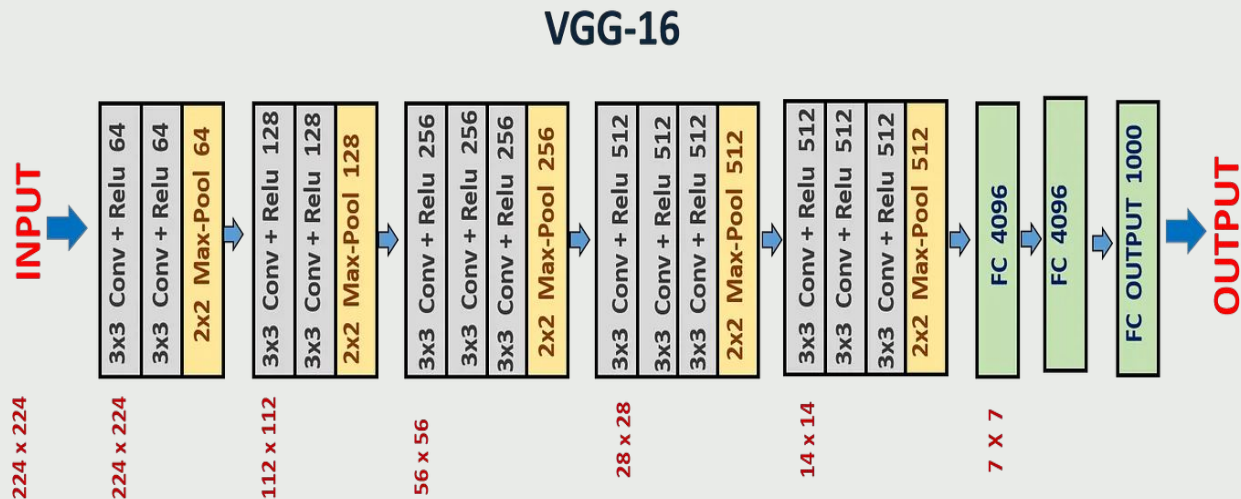
Standard Architecture:

- Stacked convolutional layers with ReLU activation
- Max pooling for spatial downsampling
- Fully connected layers for classification

Our Modification:

- Replace standard Conv2D layers with **ConvNN** or **Hybrid Branching**

*Diagram is VGG-16 instead of VGG-11

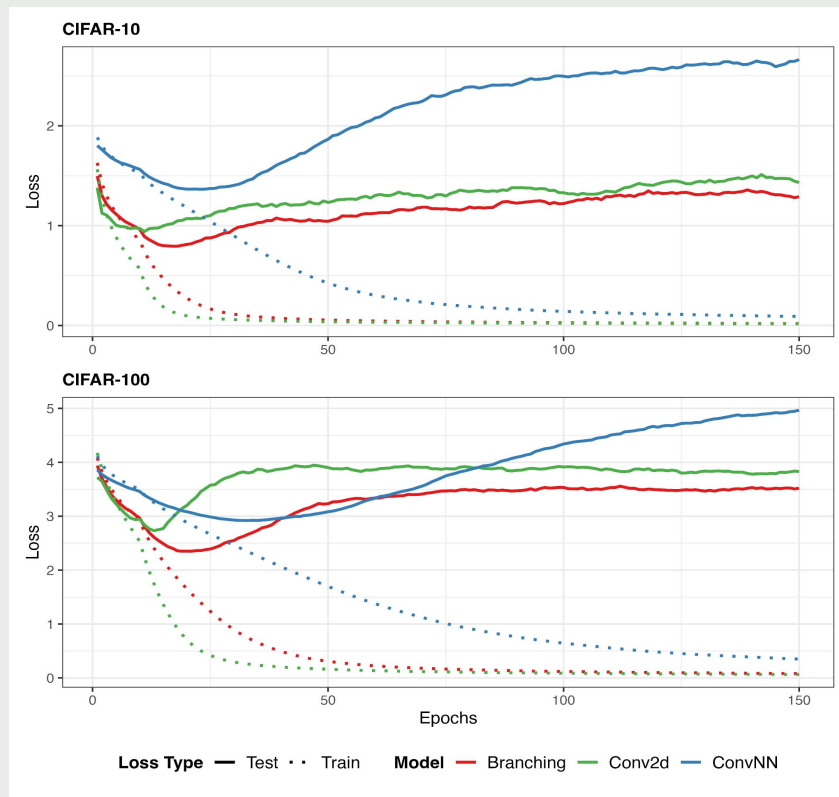


VGG-11 Results

Layer Type	CIFAR-10		CIFAR-100		GFLOPS
	Test Loss	Acc (%)	Test Loss	Acc (%)	
Conv2D	1.529	80.65	3.939	44.83	0.293
λ	0.125	1.393	81.10	3.905	45.08
	0.250	1.467	81.21	3.518	47.87
	0.375	1.422	80.95	3.578	48.34
	0.500	1.424	81.26	3.474	48.37
	0.625	1.403	80.79	3.281	49.74
	0.750	1.479	79.58	3.516	46.93
	0.875	1.332	78.18	3.906	45.72
ConvNN	2.694	54.00%	5.011	29.50%	0.325
# Params.	130.015M		130.384M		

Conv2D: Kernel Size = 3 (3x3)

ConvNN: K = 9



Vision Transformers

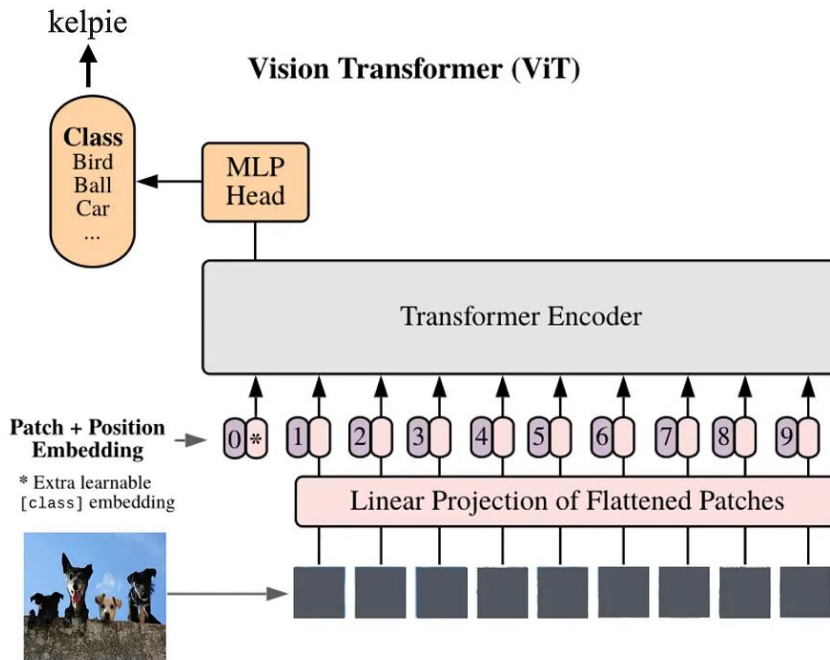
Vision Transformers (ViT)
(Dosovitskiy et al., 2021)

Architecture Overview:

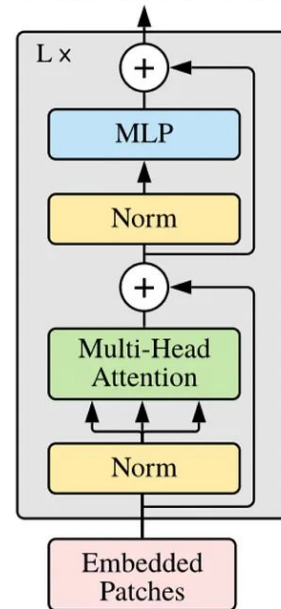
- Images are divided into fixed-size patches (16x16)
- Applied self-attention directly to image patches
- Patches are linearly embedded and fed to Transformer encoder
- Multi-Head attention captures global dependencies

Our Experiments:

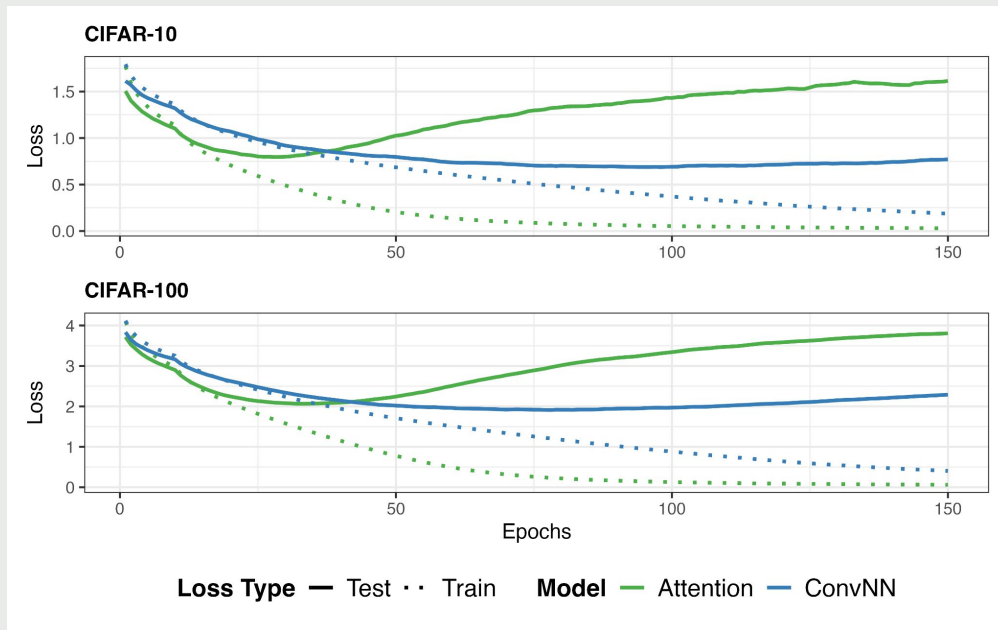
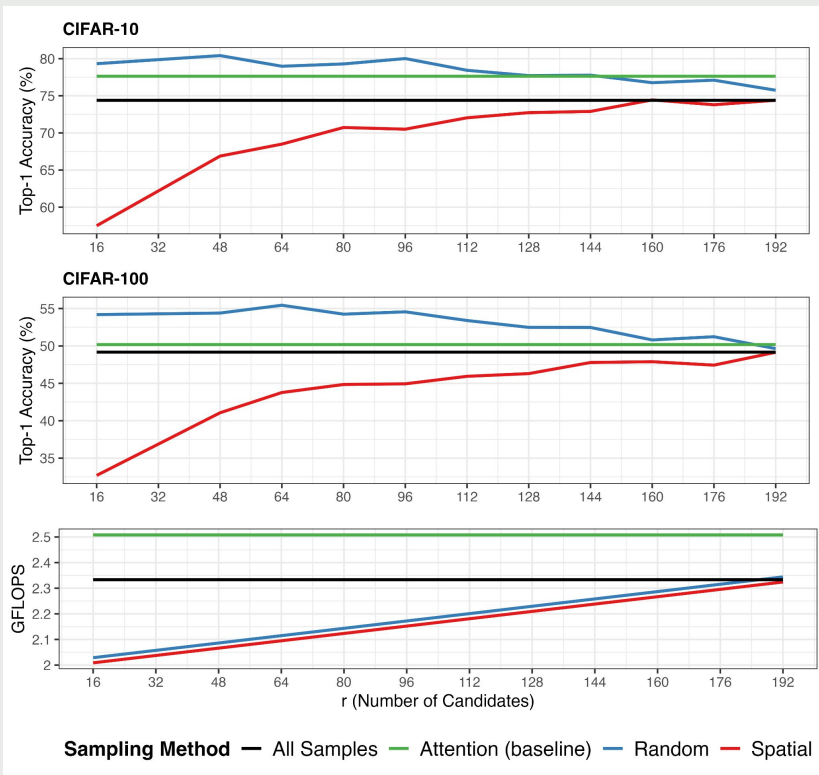
- Replace multi-head attention with **ConvNN-Attention**



Transformer Encoder



ViT-Tiny Results



ConvNN: $K = 9$
 Random Sampling: $r = r' = 32$

ViT-Tiny Results

Table 2. Test loss and accuracy for ViT-Tiny on CIFAR datasets. Bold values indicate best results; underline indicate second best.

Layer Type	CIFAR-10				CIFAR-100			
	Test Loss	Acc (%)	GFLOPS	# Params.	Test Loss	Acc (%)	GFLOPS	# Params.
Attention [29]	<u>1.558</u>	<u>77.41%</u>	2.507	<u>5.479M</u>	3.817	50.07%	2.507	<u>5.496M</u>
Local-Attention [1]	1.626	76.51%	1.991	4.299M	<u>3.688</u>	49.73%	1.991	4.317M
KVT-Attention ($k = 9$) [30]	1.775	75.77%	2.508	<u>5.479M</u>	3.883	48.21%	2.508	<u>5.496M</u>
KVT-Attention ($k = 100$) [30]	1.593	76.41%	2.508	<u>5.479M</u>	3.738	<u>50.34%</u>	2.508	<u>5.496M</u>
Sparse-Attention [6]	2.026	72.50%	2.509	<u>5.479M</u>	4.565	44.62%	2.509	<u>5.496M</u>
ConvNN (All features)	1.978	73.67%	2.333	5.500M	3.983	48.67%	2.333	5.517M
ConvNN (Random Selection)	0.779	79.82%	<u>2.037</u>	5.500M	2.341	53.60%	<u>2.037</u>	5.517M
ConvNN (Spatial Selection)	2.737	63.01%	<u>2.037</u>	5.500M	5.020	38.22%	<u>2.037</u>	5.517M

ConvNN: $K = 9$

Sampling: $r = r' = 32$

Future Work

Architectural Analysis: Compare how CNN's and Transformer's architectural components contribute to feature learning.

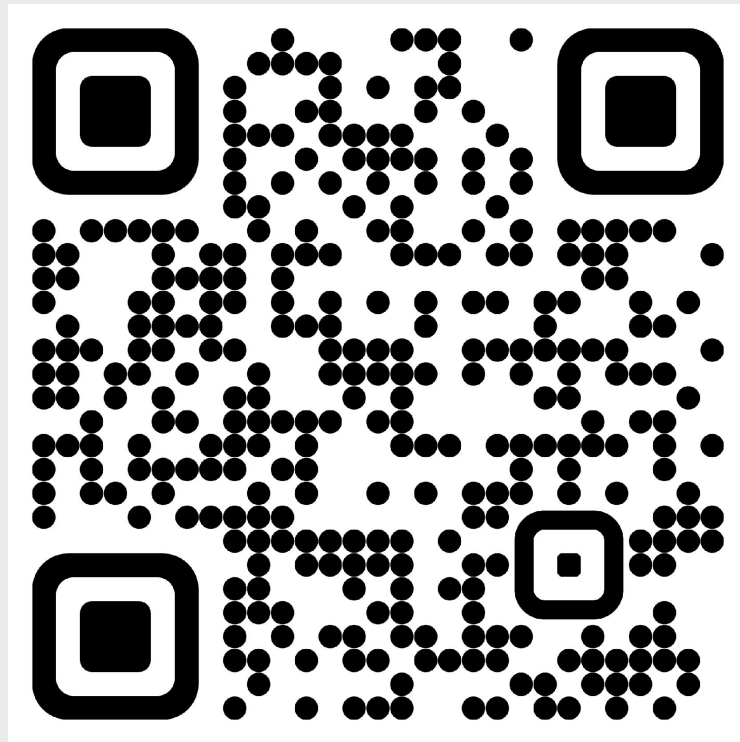
Publication: Revise and resubmit based on conference feedback (reviews expected late January).

Scaling Experiments: Extend experiments and evaluations to ImageNet-1K and other large-scale benchmarks.

Theoretical Work: Develop mathematical proof of ConvNN's generalizability framework.

Access Paper:

<https://arxiv.org/abs/2511.14137>



Thank You!